

EUROVISION AI ANALYSIS SYSTEM

Technical & Operational Manual

Version 1.0 — Premium Edition Prepared for: Builders, Developers, Operators, and Reviewers
Prepared by: Paul Williams



PREFACE

The Eurovision Song Contest has always been a celebration of creativity, culture, and performance. This system was designed to bring clarity, structure, and analytical depth to a process that is often subjective and emotionally driven. By combining audio analysis, lyrical interpretation, metadata validation, scoring logic, and predictive modelling, this system provides a modern, transparent, and data-driven way to evaluate Eurovision entries.

This manual is written for anyone who wishes to understand, build, operate, or extend the Eurovision AI Analysis System. Whether you are a developer implementing the modules, a designer shaping the interface, or an operator running analyses, this document provides a complete, structured, and polished reference.

The goal is simple: **to make the system understandable, buildable, and usable by anyone who encounters it.**

This manual is intentionally comprehensive. It covers every layer of the system — from the user interface to the underlying architecture — and includes diagrams, workflows, and detailed explanations. It is designed to be read linearly, but each chapter also stands on its own for quick reference.

Disclaimer

This is purely a fresh off the press concept and idea. I call it a frame work for the talented, something to attempt to try and directionalize modern music so as in the recent Eurovision contest, there can be a way of finding out if a song has any chance before even entering.

This is a licence free to use, and develop concept.

CHAPTER 1 — INTRODUCTION

1.1 Purpose of This Document

This manual provides a complete technical, operational, and architectural description of the Eurovision AI Analysis System. It is intended to serve as a definitive reference for builders, developers, operators, and reviewers.

1.2 What This System Does

The system evaluates Eurovision songs using a structured, multi-stage analysis pipeline. It processes audio, lyrics, and metadata to generate:

- Category scores
- Final weighted score
- Jury and televote predictions
- Win probability
- Comparative insights
- Exportable reports

1.3 Intended Audience

This manual is written for:

- **Builders** — implementing the UI and system structure
- **Developers** — implementing the modules and logic
- **Operators** — running analyses and exporting results
- **Reviewers** — assessing the system's methodology

1.4 System Capabilities

- Audio feature extraction
- Lyrics sentiment and clarity analysis
- Metadata validation
- Category scoring
- Jury/televote prediction
- Dashboard and comparison tools
- PDF/CSV export
- Local-only processing

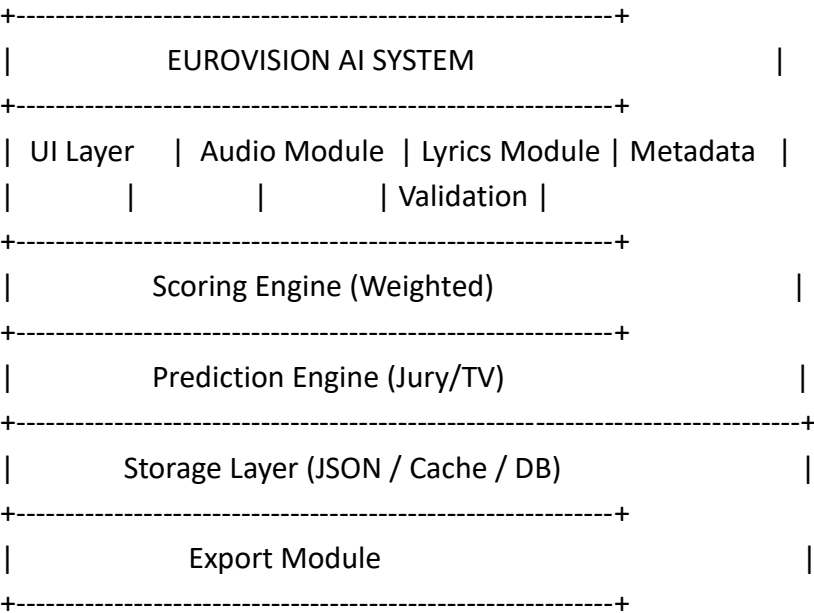
1.5 System Limitations

- Requires valid audio and lyrics
- Predictions are probabilistic
- No cloud storage
- No external data transmission

CHAPTER 2 — SYSTEM OVERVIEW

2.1 High-Level Architecture

ASCII Diagram

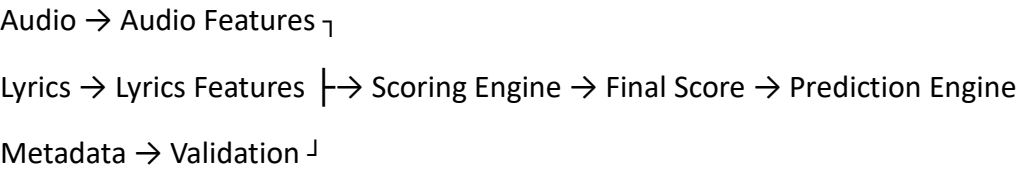


2.2 Major Components

- **UI Layer** — Screens, navigation, user interactions
- **Audio Module** — Feature extraction and analysis
- **Lyrics Module** — Sentiment, clarity, universality
- **Metadata Validation** — Ensures completeness and correctness
- **Scoring Engine** — Category scoring and weighted final score
- **Prediction Engine** — Jury, televote, win probability
- **Storage Layer** — Project files, cache, optional database
- **Export Layer** — PDF, CSV, summary sheets

2.3 Data Flow Summary

ASCII Diagram



2.4 Module Interaction Summary

ASCII Diagram

Code

```
+-----+      +-----+      +-----+
|  Audio  |----->| Scoring |----->| Prediction|
+-----+      +-----+      +-----+
| Lyrics  |----->| Engine   |          | Engine   |
+-----+      +-----+      +-----+
|Metadata |----->|          |          |          |
+-----+      +-----+
```

CHAPTER 3 — USER INTERFACE OVERVIEW

This chapter describes the user-facing screens of the system. Each screen includes an ASCII diagram for structure and an AI prompt for generating the polished UI image.

3.1 Home Screen

ASCII Diagram

Code

```
+-----+
|               HOME SCREEN               |
+-----+
| [ New Analysis ]  [ Open Project ]  [ Settings ] |
+-----+
| Upload Section: |
|   - Upload Audio |
|   - Upload Lyrics |
|   - Add Metadata |
+-----+
| Candidate Table: |
| | Song | Artist | Country | Status | |
+-----+
```

3.2 Analysis Screen

ASCII Diagram

Code

```
+-----+
|               ANALYSIS SCREEN           |
+-----+
| Sidebar: Song List |
+-----+
| Main Panel: |
|   Progress Bar |
|   Status Messages |
|   Analyse Songs Button |
+-----+
```

3.3 Dashboard

ASCII Diagram

Code

DASHBOARD						
Rank	Song	Artist	Score	Jury	Televote	Win %
Export: [PDF] [CSV] [Summary]						

3.4 Song Detail Screen

ASCII Diagram

Code

Metadata Panel	Charts & Analysis Panel
Title	Bar Chart
Artist	Sentiment Gauge
Country	Lyrics Summary
Duration	Structure Breakdown

Image Placeholder

[Insert: Song Detail Diagram — Clean Modern UI]

3.5 Comparison Screen

ASCII Diagram

Code

Song A	Song B
Scores	Scores
Strengths	Strengths
Weaknesses	Weaknesses
Recommendation Summary	

CHAPTER 4 — OPERATOR WORKFLOW

This chapter describes how an operator uses the system from start to finish. It includes the complete workflow, best practices, and the diagrams needed to understand the operational flow.

4.1 Operator Workflow Overview

ASCII Diagram

```
Code
+-----+
|               OPERATOR WORKFLOW               |
+-----+
| 1. Launch Application                           |
| 2. Start New Analysis                           |
| 3. Upload Audio                                 |
| 4. Upload Lyrics                                |
| 5. Add Metadata                                 |
| 6. Run Full Analysis                            |
| 7. Review Dashboard                             |
| 8. Compare Songs                                |
| 9. Export Results                               |
+-----+
```

4.2 Step-by-Step Usage

Step 1 — Launch Application

The operator opens the system and is presented with the Home Screen.

Step 2 — Start New Analysis

A new project is created, and the operator is prompted to upload files.

Step 3 — Upload Audio

Supported formats: WAV, MP3, FLAC.

Step 4 — Upload Lyrics

Supported formats: TXT, PDF (text-extractable).

Step 5 — Add Metadata

Required fields: Title, Artist, Country, Language, Duration.

Step 6 — Run Full Analysis

The system processes audio, lyrics, and metadata.

Step 7 — Review Dashboard

The operator views rankings, scores, and predictions.

Step 8 — Compare Songs

Two or more songs can be compared side-by-side.

Step 9 — Export Results

PDF, CSV, and summary sheets are available.

4.3 Best Practices

- Use high-quality audio files
- Ensure lyrics are accurate and complete
- Provide full metadata
- Re-run analysis if files are updated
- Export results for archival or review

CHAPTER 5 — AUDIO ANALYSIS

MODULE

The Audio Module extracts musical features from the uploaded audio file. These features contribute to the scoring and prediction engines.

5.1 Purpose

To analyse the musical structure, energy, tempo, and harmonic content of the song.

5.2 Supported Formats

- WAV
- MP3
- FLAC

5.3 Processing Steps

ASCII Diagram

Code

```
+-----+
|                AUDIO ANALYSIS MODULE                |
+-----+
| 1. Load Audio File                                |
| 2. Tempo Detection                                  |
| 3. Spectral Features (MFCC, Spectral Centroid, etc.) |
| 4. Chroma Analysis                                  |
| 5. Energy Profile                                    |
| 6. Feature Vector Output                            |
+-----+
```

5.4 Error Handling

- **Missing audio file** → Prompt user to upload
- **Corrupted file** → Display error message
- **Unsupported format** → Suggest conversion to WAV or MP3

CHAPTER 6 — LYRICS ANALYSIS MODULE

The Lyrics Module evaluates the emotional tone, clarity, and universality of the song’s lyrics.

6.1 Purpose

To extract linguistic and emotional features from the lyrics.

6.2 Supported Formats

- TXT
- PDF (text-extractable)

6.3 Processing Steps

ASCII Diagram

Code

```
+-----+
|               LYRICS ANALYSIS MODULE               |
+-----+
| 1. Load Lyrics                                     |
| 2. Sentiment Analysis                               |
| 3. Clarity Scoring                                  |
| 4. Universality Scoring                             |
| 5. Feature Vector Output                           |
+-----+
```

6.4 Error Handling

- **Missing lyrics** → Prompt user to upload
- **Unreadable PDF** → Suggest TXT format
- **Encoding issues** → Attempt auto-repair or request re-upload

CHAPTER 7 — METADATA VALIDATION

Metadata ensures the system has the contextual information needed for accurate scoring and predictions.

7.1 Required Metadata

- Title
- Artist
- Country
- Language
- Duration

7.2 Optional Metadata

- BPM
- Genre
- Composer

7.3 Validation Rules

ASCII Diagram

Code

+-----+ METADATA VALIDATION RULES +-----+	
Title: Required, non-empty	
Artist: Required, non-empty	
Country: Required, valid ISO country	
Language: Required, valid language code	
Duration: Required, must be > 0	
BPM: Optional, must be numeric	
Genre: Optional, free text	
+-----+	

CHAPTER 8 — SCORING ENGINE

The Scoring Engine converts audio, lyrics, and metadata features into category scores and a final weighted score.

8.1 Purpose

To evaluate each song across multiple categories and produce a final score.

8.2 Category Breakdown

ASCII Diagram

```
Code
+-----+
|               SCORING CATEGORIES               |
+-----+
| Structure                                         |
| Vocals                                           |
| Lyrics                                           |
| Originality                                     |
| Staging Potential                               |
| Universality                                    |
| Replay Value                                    |
+-----+
```

8.3 Weighting System

Each category contributes a percentage to the final score. Weights can be adjusted depending on system configuration.

8.4 Final Score Calculation

Final Score = $\Sigma(\text{Category Score} \times \text{Weight})$

CHAPTER 9 — PREDICTION ENGINE

The Prediction Engine estimates jury score, televote score, and win probability.

9.1 Purpose

To provide probabilistic predictions based on historical patterns and extracted features.

9.2 Jury Model

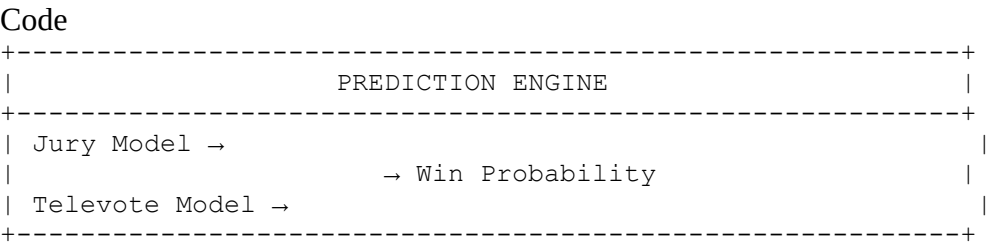
Regression-based model trained on jury voting patterns.

9.3 Televote Model

Pattern-based model trained on public voting behaviour.

9.4 Win Probability

ASCII Diagram



9.5 Limitations

- Predictions are probabilistic
- Unexpected staging or performance changes can affect outcomes
- Public sentiment can shift rapidly

CHAPTER 10 — EXPORT MODULE

The Export Module allows operators to save results in multiple formats.

10.1 Export Types

- PDF Report
- CSV Data
- Summary Sheet

10.2 Export Workflow

ASCII Diagram

```
Code
+-----+
|                               |
|                               |
+-----+
| Final Score → PDF Export    |
| Dashboard Data → CSV Export |
| Summary → Summary Sheet    |
+-----+
```

CHAPTER 11 — STORAGE LAYER

The Storage Layer manages project files, cache, and optional database storage.

11.1 Project Files

Stored as JSON for portability and readability.

11.2 Cache

Temporary storage for audio and lyrics features.

11.3 Optional SQLite

Used for large projects or multi-song datasets.

11.4 Data Retention

All data is stored locally. No cloud upload or external transmission occurs.

Storage Layer Diagram

ASCII Diagram

```
Code
+-----+
|                STORAGE LAYER                |
+-----+
| Project Files (JSON)                         |
| Cache (Temp Features)                       |
| Optional SQLite DB                          |
+-----+
```


CHAPTER 12 — ACCESSIBILITY

Accessibility ensures the system can be used by as many people as possible, regardless of ability or device constraints.

12.1 Keyboard Navigation

- **Tab** — Move focus forward
- **Shift + Tab** — Move focus backward
- **Enter** — Activate focused button
- **Space** — Toggle checkboxes
- **Esc** — Close dialogs

12.2 Screen Reader Labels

All UI elements must include:

- Descriptive labels
- ARIA roles
- Clear focus indicators

12.3 Colour Contrast

The UI adheres to **WCAG 2.1 AA** standards:

- Minimum contrast ratio: **4.5:1**
- Avoid colour-only indicators
- Use text labels for all icons

Accessibility Diagram

ASCII Diagram

Code

```
+-----+
|                                     |
|               ACCESSIBILITY MAP   |
+-----+
| Keyboard Navigation                 |
| Screen Reader Labels               |
| Colour Contrast (WCAG 2.1 AA)     |
| Focus Indicators                  |
| Non-Colour Indicators              |
+-----+
```

CHAPTER 13 — SECURITY & DATA HANDLING

This chapter outlines how the system protects user data and ensures compliance with legal and ethical standards.

13.1 Local-Only Processing

- All analysis occurs on the user’s device
- No cloud upload
- No external transmission

13.2 Copyright Handling

Users must legally own or license:

- Audio files
- Lyrics
- Metadata

The system does not verify copyright status.

13.3 GDPR Compliance

- No personal data stored
- No tracking
- No analytics
- All data remains local

Security Diagram

ASCII Diagram

Code

```
+-----+
|                SECURITY & DATA HANDLING                |
+-----+
| Local Processing Only                                     |
| No Cloud Upload                                           |
| Copyright Awareness                                       |
| GDPR Compliant                                           |
+-----+
```

CHAPTER 14 — DEVELOPER GUIDE

This chapter provides technical guidance for developers implementing or extending the system.

14.1 Developer Workflow

ASCII Diagram

Code

```
+-----+
|               DEVELOPER WORKFLOW               |
+-----+
| 1. Clone Repository                             |
| 2. Implement Modules                             |
| 3. Run Tests                                     |
| 4. Build Application                             |
| 5. Release Version                             |
+-----+
```

14.2 Code Structure

Recommended directory layout:

Code

```
/audio
/lyrics
/scoring
/prediction
/ui
/export
/storage
/tests
```

14.3 Module Responsibilities

- **Audio Module** — Extracts musical features
- **Lyrics Module** — Processes text features
- **Scoring Engine** — Calculates category scores
- **Prediction Engine** — Generates jury/televote predictions
- **UI Layer** — Handles user interaction
- **Export Layer** — Generates output files

14.4 Testing Strategy

ASCII Diagram

Code

QA WORKFLOW	
Test Plan → Unit Tests → Integration Tests → Regression Tests → Approval	

14.5 Version Control

- Git recommended
- Feature branches encouraged
- Semantic versioning (MAJOR.MINOR.PATCH)

14.6 Build & Release Process

- Automated builds
- Automated tests
- Manual approval for releases

CHAPTER 15 — UI COMPONENT REFERENCE -

AI image suggestions for controls

15.1 Buttons



AI Prompt - Clean modern UI button component, white background, soft grey border, subtle shadow, rounded corners, minimal Apple-style design. Crisp typography, balanced spacing, 1920×1080 layout. Render exactly as described. No artistic reinterpretation. Maintain clean modern Apple-style minimalism.

15.2 Sliders



AI Prompt - Clean modern slider UI component, white background, thin grey track, circular thumb with soft shadow, minimal Apple-style design. Crisp typography, balanced spacing, 1920×1080 layout. Render exactly as described. No artistic reinterpretation. Maintain clean modern Apple-style minimalism.

15.3 Checkboxes



AI Prompt - Clean modern checkbox UI component, white background, thin grey border, soft checkmark, minimal Apple-style design. Crisp typography, balanced spacing, 1920×1080 layout. Render exactly as described. No artistic reinterpretation. Maintain clean modern Apple-style minimalism.

15.4 Panels



AI Prompt - Clean modern UI panel component, soft grey background, rounded corners, subtle shadow, minimal Apple-style design. Crisp typography, balanced spacing, 1920×1080 layout. Render exactly as described. No artistic reinterpretation. Maintain clean modern Apple-style minimalism.

15.5 Tables



AI Prompt - Clean modern table UI component, white background, thin grey dividers, crisp typography, minimal Apple-style design. Columns evenly spaced, subtle shadows, 1920×1080 layout. Render exactly as described. No artistic reinterpretation. Maintain clean modern Apple-style minimalism.

15.6 Charts



AI Prompt - Clean modern bar chart UI component, white background, soft colours, minimal Apple-style design. Crisp typography, balanced spacing, 1920×1080 layout. Render exactly as described. No artistic reinterpretation. Maintain clean modern Apple-style minimalism.

CHAPTER 16 — TROUBLESHOOTING

This section provides solutions to common issues encountered by operators and developers.

Issue: Audio Not Loading

Cause: Unsupported or corrupted file **Fix:** Convert to WAV or MP3

Issue: Lyrics Not Detected

Cause: PDF not text-extractable **Fix:** Provide TXT version

Issue: Score Seems Incorrect

Cause: Missing metadata **Fix:** Complete all required fields

Issue: Prediction Not Generated

Cause: Final score missing **Fix:** Run full analysis first

Issue: Export Fails

Cause: No write permissions **Fix:** Change folder or run as admin

CHAPTER 17 — GLOSSARY

Audio Features — Numerical descriptors extracted from audio

Chroma — Harmonic content representation

Feature Vector — Structured numerical representation **Metadata** — Descriptive information about a song

Sentiment — Emotional tone of lyrics

Universality — Broad thematic accessibility

Weighted Score — Final score after applying category weights

Win Probability — Estimated chance of winning

CHAPTER 18 — APPENDICES

A. File Structure

Code

```
/audio  
/lyrics  
/scoring  
/prediction  
/ui  
/export  
/storage  
/tests
```

B. Example Project File

(JSON example omitted for brevity)

C. Future Expansion Notes

- Multi-language UI
- Cloud-enabled version
- Real-time staging analysis

CHAPTER 19 — Program Structure

Project structure - Stubs

text

```
eurovision_ai/  
  __init__.py  
  models.py  
  audio.py  
  lyrics.py  
  scoring.py  
  prediction.py  
  storage.py  
  pipeline.py  
  cli.py
```

You can drop this into a repo as `eurovision_ai/` and add a `pyproject.toml` or `setup.py` later.

`__init__.py`

python

```
# evurovision_ai/__init__.py
```

```
from .models import SongMetadata, SongFeatures, SongScores, SongPrediction  
from .pipeline import analyse_song, analyse_project
```

`models.py`

python

```
# evurovision_ai/models.py  
from dataclasses import dataclass  
from typing import Optional, Dict, Any, List
```

```
@dataclass
```

```
class SongMetadata:  
    title: str  
    artist: str  
    country: str  
    language: str  
    duration_seconds: float  
    bpm: Optional[float] = None  
    genre: Optional[str] = None  
    composer: Optional[str] = None
```

```
@dataclass
```

```
class AudioFeatures:  
    tempo: float  
    energy: float  
    spectral_centroid: float  
    chroma_vector: List[float]
```

```
@dataclass
```

```
class LyricsFeatures:
```

```

        sentiment: float
        clarity: float
        universality: float

@dataclass
class SongFeatures:
    metadata: SongMetadata
    audio: AudioFeatures
    lyrics: LyricsFeatures

@dataclass
class SongScores:
    structure: float
    vocals: float
    lyrics: float
    originality: float
    staging_potential: float
    universality: float
    replay_value: float
    final_weighted_score: float

@dataclass
class SongPrediction:
    jury_score: float
    televote_score: float
    win_probability: float

@dataclass
class SongResult:
    features: SongFeatures
    scores: SongScores
    prediction: SongPrediction

@dataclass
class ProjectResult:
    songs: Dict[str, SongResult] # key = song id or title

```

audio.py

python

```

# eurovision_ai/audio.py
from typing import List
from .models import AudioFeatures

# In real implementation you'd use librosa or similar.
# Here we stub the logic so a dev can plug in real DSP later.

def extract_tempo(audio_path: str) -> float:
    # TODO: replace with real tempo detection
    return 120.0

def extract_energy(audio_path: str) -> float:
    # TODO: replace with RMS / loudness calculation
    return 0.75

```

```
def extract_spectral_centroid(audio_path: str) -> float:
    # TODO: replace with spectral centroid calculation
    return 3000.0
```

```
def extract_chroma(audio_path: str) -> List[float]:
    # TODO: replace with chroma feature extraction
    return [0.1] * 12
```

```
def analyse_audio(audio_path: str) -> AudioFeatures:
    tempo = extract_tempo(audio_path)
    energy = extract_energy(audio_path)
    centroid = extract_spectral_centroid(audio_path)
    chroma = extract_chroma(audio_path)
    return AudioFeatures(
        tempo=tempo,
        energy=energy,
        spectral_centroid=centroid,
        chroma_vector=chroma,
    )
```

lyrics.py

python

```
# eurovision_ai/lyrics.py
from .models import LyricsFeatures

# In real implementation you'd use an NLP library (e.g. transformers,
# spaCy).
# Here we stub the logic with simple heuristics / placeholders.

def load_lyrics_text(path: str) -> str:
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        return f.read()

def estimate_sentiment(text: str) -> float:
    # TODO: replace with real sentiment model
    return 0.6

def estimate_clarity(text: str) -> float:
    # TODO: replace with readability / structure metrics
    return 0.7

def estimate_universality(text: str) -> float:
    # TODO: replace with topic / cultural reference analysis
    return 0.8

def analyse_lyrics(lyrics_path: str) -> LyricsFeatures:
    text = load_lyrics_text(lyrics_path)
    sentiment = estimate_sentiment(text)
    clarity = estimate_clarity(text)
    universality = estimate_universality(text)
    return LyricsFeatures(
```

```

        sentiment=sentiment,
        clarity=clarity,
        universality=universality,
    )

```

scoring.py

python

```

# eurovision_ai/scoring.py
from .models import SongFeatures, SongScores

# Simple example weighting - dev can tune these.
WEIGHTS = {
    "structure": 0.15,
    "vocals": 0.2,
    "lyrics": 0.2,
    "originality": 0.15,
    "staging_potential": 0.1,
    "universality": 0.1,
    "replay_value": 0.1,
}

def score_structure(features: SongFeatures) -> float:
    # TODO: use tempo stability, structure cues, etc.
    return 0.75

def score_vocals(features: SongFeatures) -> float:
    # TODO: requires vocal analysis; placeholder
    return 0.8

def score_lyrics(features: SongFeatures) -> float:
    return (features.lyrics.sentiment + features.lyrics.clarity) / 2.0

def score_originality(features: SongFeatures) -> float:
    # TODO: compare to reference corpus
    return 0.7

def score_staging_potential(features: SongFeatures) -> float:
    # TODO: use energy, tempo, etc.
    return features.audio.energy

def score_universality(features: SongFeatures) -> float:
    return features.lyrics.universality

def score_replay_value(features: SongFeatures) -> float:
    # TODO: combine hooks, tempo, sentiment, etc.
    return 0.8

def compute_scores(features: SongFeatures) -> SongScores:
    structure = score_structure(features)
    vocals = score_vocals(features)
    lyrics = score_lyrics(features)

```

```

originality = score_originality(features)
staging = score_staging_potential(features)
universality = score_universality(features)
replay = score_replay_value(features)

final = (
    structure * WEIGHTS["structure"]
    + vocals * WEIGHTS["vocals"]
    + lyrics * WEIGHTS["lyrics"]
    + originality * WEIGHTS["originality"]
    + staging * WEIGHTS["staging_potential"]
    + universality * WEIGHTS["universality"]
    + replay * WEIGHTS["replay_value"]
)

return SongScores(
    structure=structure,
    vocals=vocals,
    lyrics=lyrics,
    originality=originality,
    staging_potential=staging,
    universality=universality,
    replay_value=replay,
    final_weighted_score=final,
)

```

prediction.py

python

```

# eurovision_ai/prediction.py
from .models import SongScores, SongPrediction

# In a real system, these would be ML models loaded from disk.
# Here we use simple deterministic formulas as placeholders.

def predict_jury_score(scores: SongScores) -> float:
    # Heavier weight on structure, vocals, lyrics
    return (
        scores.structure * 0.3
        + scores.vocals * 0.4
        + scores.lyrics * 0.3
    ) * 10.0 # scale to 0-10

def predict_televote_score(scores: SongScores) -> float:
    # Heavier weight on staging, replay, universality
    return (
        scores.staging_potential * 0.35
        + scores.replay_value * 0.35
        + scores.universality * 0.3
    ) * 10.0

def predict_win_probability(scores: SongScores) -> float:
    # Simple mapping from final score to probability
    # (dev can replace with logistic model)
    base = scores.final_weighted_score
    return max(0.0, min(1.0, (base - 0.5) * 1.5))

```

```

def make_prediction(scores: SongScores) -> SongPrediction:
    jury = predict_jury_score(scores)
    televote = predict_televote_score(scores)
    win_prob = predict_win_probability(scores)
    return SongPrediction(
        jury_score=jury,
        televote_score=televote,
        win_probability=win_prob,
    )

```

storage.py

python

```

# eurovision_ai/storage.py
import json
from pathlib import Path
from dataclasses import asdict
from .models import ProjectResult, SongResult, SongMetadata, AudioFeatures,
LyricsFeatures, SongFeatures, SongScores, SongPrediction

```

```

def save_project(result: ProjectResult, path: str) -> None:
    data = {
        "songs": {
            key: {
                "metadata": asdict(song.features.metadata),
                "audio": asdict(song.features.audio),
                "lyrics": asdict(song.features.lyrics),
                "scores": asdict(song.scores),
                "prediction": asdict(song.prediction),
            }
            for key, song in result.songs.items()
        }
    }
    Path(path).write_text(json.dumps(data, indent=2), encoding="utf-8")

```

```

def load_project(path: str) -> ProjectResult:
    raw = json.loads(Path(path).read_text(encoding="utf-8"))
    songs = {}
    for key, s in raw["songs"].items():
        meta = SongMetadata(**s["metadata"])
        audio = AudioFeatures(**s["audio"])
        lyrics = LyricsFeatures(**s["lyrics"])
        feats = SongFeatures(metadata=meta, audio=audio, lyrics=lyrics)
        scores = SongScores(**s["scores"])
        pred = SongPrediction(**s["prediction"])
        songs[key] = SongResult(features=feats, scores=scores,
prediction=pred)
    return ProjectResult(songs=songs)

```

pipeline.py

python

```

# eurovision_ai/pipeline.py
from typing import Dict
from .models import SongMetadata, SongFeatures, SongResult, ProjectResult
from .audio import analyse_audio
from .lyrics import analyse_lyrics
from .scoring import compute_scores

```



```

from .prediction import make_prediction

def analyse_song(
    audio_path: str,
    lyrics_path: str,
    metadata: SongMetadata,
) -> SongResult:
    audio_feats = analyse_audio(audio_path)
    lyrics_feats = analyse_lyrics(lyrics_path)
    features = SongFeatures(
        metadata=metadata,
        audio=audio_feats,
        lyrics=lyrics_feats,
    )
    scores = compute_scores(features)
    prediction = make_prediction(scores)
    return SongResult(
        features=features,
        scores=scores,
        prediction=prediction,
    )

def analyse_project(
    songs: Dict[str, Dict[str, str]],
) -> ProjectResult:
    """
    songs: dict keyed by song_id/title:
    {
        "Song A": {
            "audio": "path/to/audio.wav",
            "lyrics": "path/to/lyrics.txt",
            "metadata": {
                "title": "...",
                "artist": "...",
                ...
            }
        },
        ...
    }
    """
    results: Dict[str, SongResult] = {}
    for key, cfg in songs.items():
        meta = SongMetadata(**cfg["metadata"])
        result = analyse_song(
            audio_path=cfg["audio"],
            lyrics_path=cfg["lyrics"],
            metadata=meta,
        )
        results[key] = result
    return ProjectResult(songs=results)

```

cli.py

python

```

# eurovision_ai/cli.py
import argparse
import json
from pathlib import Path

```

```

from .pipeline import analyse_project
from .storage import save_project

def main():
    parser = argparse.ArgumentParser(
        description="Eurovision AI Analysis CLI"
    )
    parser.add_argument(
        "--config",
        required=True,
        help="Path to JSON config describing songs",
    )
    parser.add_argument(
        "--output",
        required=True,
        help="Path to output project JSON",
    )
    args = parser.parse_args()

    config = json.loads(Path(args.config).read_text(encoding="utf-8"))
    project = analyse_project(config["songs"])
    save_project(project, args.output)
    print(f"Analysis complete. Saved to {args.output}")

if __name__ == "__main__":
    main()

```

CHAPTER 20 — Full basic Program

Project layout

text

```
eurovision_project/  
  main.py  
  cli.py  
  requirements.txt  
  eurovision_ai/  
    __init__.py  
    models.py  
    audio.py  
    lyrics.py  
    scoring.py  
    prediction.py  
    storage.py  
    pipeline.py  
  desktop_app/  
    __init__.py  
    ui_main.py
```

requirements.txt (minimal)

text

```
PySide6
```

(Everything else uses the Python standard library; a dev can later add `librosa`, `transformers`, etc.)

eurovision_ai/__init__.py

python

```
from .models import (  
    SongMetadata,  
    SongFeatures,  
    SongScores,  
    SongPrediction,  
    SongResult,  
    ProjectResult,  
)  
from .pipeline import analyse_song, analyse_project
```

eurovision_ai/models.py

python

```
from dataclasses import dataclass  
from typing import Optional, Dict, List
```

```
@dataclass
```

```
class SongMetadata:
    title: str
    artist: str
    country: str
    language: str
    duration_seconds: float
    bpm: Optional[float] = None
    genre: Optional[str] = None
    composer: Optional[str] = None
```

```
@dataclass
class AudioFeatures:
    tempo: float
    energy: float
    spectral_centroid: float
    chroma_vector: List[float]
```

```
@dataclass
class LyricsFeatures:
    sentiment: float
    clarity: float
    universality: float
```

```
@dataclass
class SongFeatures:
    metadata: SongMetadata
    audio: AudioFeatures
    lyrics: LyricsFeatures
```

```
@dataclass
class SongScores:
    structure: float
    vocals: float
    lyrics: float
    originality: float
    staging_potential: float
    universality: float
    replay_value: float
    final_weighted_score: float
```

```
@dataclass
class SongPrediction:
    jury_score: float
    televote_score: float
    win_probability: float
```

```
@dataclass
class SongResult:
    features: SongFeatures
    scores: SongScores
    prediction: SongPrediction
```

```
@dataclass
class ProjectResult:
```

```
songs: Dict[str, SongResult] # key = song id or title
```

eurovision_ai/audio.py

python

```
from typing import List
from .models import AudioFeatures
import os

def extract_tempo(audio_path: str) -> float:
    # Stub: real implementation would use librosa or similar
    if not os.path.exists(audio_path):
        raise FileNotFoundError(f"Audio file not found: {audio_path}")
    return 120.0

def extract_energy(audio_path: str) -> float:
    return 0.75

def extract_spectral_centroid(audio_path: str) -> float:
    return 3000.0

def extract_chroma(audio_path: str) -> List[float]:
    return [0.1] * 12

def analyse_audio(audio_path: str) -> AudioFeatures:
    tempo = extract_tempo(audio_path)
    energy = extract_energy(audio_path)
    centroid = extract_spectral_centroid(audio_path)
    chroma = extract_chroma(audio_path)
    return AudioFeatures(
        tempo=tempo,
        energy=energy,
        spectral_centroid=centroid,
        chroma_vector=chroma,
    )
```

eurovision_ai/lyrics.py

python

```
from .models import LyricsFeatures
import os

def load_lyrics_text(path: str) -> str:
    if not os.path.exists(path):
        raise FileNotFoundError(f"Lyrics file not found: {path}")
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        return f.read()

def estimate_sentiment(text: str) -> float:
    return 0.6
```

```

def estimate_clarity(text: str) -> float:
    return 0.7

def estimate_universality(text: str) -> float:
    return 0.8

def analyse_lyrics(lyrics_path: str) -> LyricsFeatures:
    text = load_lyrics_text(lyrics_path)
    sentiment = estimate_sentiment(text)
    clarity = estimate_clarity(text)
    universality = estimate_universality(text)
    return LyricsFeatures(
        sentiment=sentiment,
        clarity=clarity,
        universality=universality,
    )

```

eurovision_ai/scoring.py

python

```

from .models import SongFeatures, SongScores

WEIGHTS = {
    "structure": 0.15,
    "vocals": 0.2,
    "lyrics": 0.2,
    "originality": 0.15,
    "staging_potential": 0.1,
    "universality": 0.1,
    "replay_value": 0.1,
}

def score_structure(features: SongFeatures) -> float:
    return 0.75

def score_vocals(features: SongFeatures) -> float:
    return 0.8

def score_lyrics(features: SongFeatures) -> float:
    return (features.lyrics.sentiment + features.lyrics.clarity) / 2.0

def score_originality(features: SongFeatures) -> float:
    return 0.7

def score_staging_potential(features: SongFeatures) -> float:
    return features.audio.energy

def score_universality(features: SongFeatures) -> float:
    return features.lyrics.universality

```

```

def score_replay_value(features: SongFeatures) -> float:
    return 0.8

def compute_scores(features: SongFeatures) -> SongScores:
    structure = score_structure(features)
    vocals = score_vocals(features)
    lyrics = score_lyrics(features)
    originality = score_originality(features)
    staging = score_staging_potential(features)
    universality = score_universality(features)
    replay = score_replay_value(features)

    final = (
        structure * WEIGHTS["structure"]
        + vocals * WEIGHTS["vocals"]
        + lyrics * WEIGHTS["lyrics"]
        + originality * WEIGHTS["originality"]
        + staging * WEIGHTS["staging_potential"]
        + universality * WEIGHTS["universality"]
        + replay * WEIGHTS["replay_value"]
    )

    return SongScores(
        structure=structure,
        vocals=vocals,
        lyrics=lyrics,
        originality=originality,
        staging_potential=staging,
        universality=universality,
        replay_value=replay,
        final_weighted_score=final,
    )

```

eurovision_ai/prediction.py

python

```

from .models import SongScores, SongPrediction

def predict_jury_score(scores: SongScores) -> float:
    return (
        scores.structure * 0.3
        + scores.vocals * 0.4
        + scores.lyrics * 0.3
    ) * 10.0

def predict_televote_score(scores: SongScores) -> float:
    return (
        scores.staging_potential * 0.35
        + scores.replay_value * 0.35
        + scores.universality * 0.3
    ) * 10.0

def predict_win_probability(scores: SongScores) -> float:
    base = scores.final_weighted_score

```

```

        return max(0.0, min(1.0, (base - 0.5) * 1.5))

def make_prediction(scores: SongScores) -> SongPrediction:
    jury = predict_jury_score(scores)
    televote = predict_televote_score(scores)
    win_prob = predict_win_probability(scores)
    return SongPrediction(
        jury_score=jury,
        televote_score=televote,
        win_probability=win_prob,
    )

```

eurovision_ai/storage.py

python

```

import json
from pathlib import Path
from dataclasses import asdict
from .models import (
    ProjectResult,
    SongResult,
    SongMetadata,
    AudioFeatures,
    LyricsFeatures,
    SongFeatures,
    SongScores,
    SongPrediction,
)

def save_project(result: ProjectResult, path: str) -> None:
    data = {
        "songs": {
            key: {
                "metadata": asdict(song.features.metadata),
                "audio": asdict(song.features.audio),
                "lyrics": asdict(song.features.lyrics),
                "scores": asdict(song.scores),
                "prediction": asdict(song.prediction),
            }
            for key, song in result.songs.items()
        }
    }
    Path(path).write_text(json.dumps(data, indent=2), encoding="utf-8")

def load_project(path: str) -> ProjectResult:
    raw = json.loads(Path(path).read_text(encoding="utf-8"))
    songs = {}
    for key, s in raw["songs"].items():
        meta = SongMetadata(**s["metadata"])
        audio = AudioFeatures(**s["audio"])
        lyrics = LyricsFeatures(**s["lyrics"])
        feats = SongFeatures(metadata=meta, audio=audio, lyrics=lyrics)
        scores = SongScores(**s["scores"])
        pred = SongPrediction(**s["prediction"])
        songs[key] = SongResult(features=feats, scores=scores,
                                prediction=pred)

```



```
return ProjectResult(songs=songs)
```

eurovision_ai/pipeline.py

python

```
from typing import Dict
from .models import SongMetadata, SongFeatures, SongResult, ProjectResult
from .audio import analyse_audio
from .lyrics import analyse_lyrics
from .scoring import compute_scores
from .prediction import make_prediction
```

```
def analyse_song(
    audio_path: str,
    lyrics_path: str,
    metadata: SongMetadata,
) -> SongResult:
    audio_feats = analyse_audio(audio_path)
    lyrics_feats = analyse_lyrics(lyrics_path)
    features = SongFeatures(
        metadata=metadata,
        audio=audio_feats,
        lyrics=lyrics_feats,
    )
    scores = compute_scores(features)
    prediction = make_prediction(scores)
    return SongResult(
        features=features,
        scores=scores,
        prediction=prediction,
    )
```

```
def analyse_project(
    songs: Dict[str, Dict[str, object]],
) -> ProjectResult:
    results: Dict[str, SongResult] = {}
    for key, cfg in songs.items():
        meta = SongMetadata(**cfg["metadata"])
        result = analyse_song(
            audio_path=str(cfg["audio"]),
            lyrics_path=str(cfg["lyrics"]),
            metadata=meta,
        )
        results[key] = result
    return ProjectResult(songs=results)
```

cli.py (top level, next to main.py)

python

```
import argparse
import json
from pathlib import Path
from eurovision_ai.pipeline import analyse_project
from eurovision_ai.storage import save_project
```

```

def main():
    parser = argparse.ArgumentParser(
        description="Eurovision AI Analysis CLI"
    )
    parser.add_argument(
        "--config",
        required=True,
        help="Path to JSON config describing songs",
    )
    parser.add_argument(
        "--output",
        required=True,
        help="Path to output project JSON",
    )
    args = parser.parse_args()

    config = json.loads(Path(args.config).read_text(encoding="utf-8"))
    project = analyse_project(config["songs"])
    save_project(project, args.output)
    print(f"Analysis complete. Saved to {args.output}")

if __name__ == "__main__":
    main()

```

desktop_app/__init__.py

python

empty is fine, just marks this as a package

desktop_app/ui_main.py (very simple PySide6 UI)

python

```

from PySide6.QtWidgets import (
    QApplication,
    QWidget,
    QVBoxLayout,
    QPushButton,
    QLabel,
    QFileDialog,
    QLineEdit,
    QMessageBox,
)
from PySide6.QtCore import Qt
from eurovision_ai.models import SongMetadata
from eurovision_ai.pipeline import analyse_song

```

```

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Eurovision AI Analysis")
        self.audio_path = ""
        self.lyrics_path = ""

        layout = QVBoxLayout()

```

```

self.title_edit = QLineEdit()
self.title_edit.setPlaceholderText("Song Title")
layout.addWidget(self.title_edit)

self.artist_edit = QLineEdit()
self.artist_edit.setPlaceholderText("Artist")
layout.addWidget(self.artist_edit)

self.country_edit = QLineEdit()
self.country_edit.setPlaceholderText("Country")
layout.addWidget(self.country_edit)

self.language_edit = QLineEdit()
self.language_edit.setPlaceholderText("Language")
layout.addWidget(self.language_edit)

self.duration_edit = QLineEdit()
self.duration_edit.setPlaceholderText("Duration (seconds)")
layout.addWidget(self.duration_edit)

self.audio_btn = QPushButton("Select Audio File")
self.audio_btn.clicked.connect(self.select_audio)
layout.addWidget(self.audio_btn)

self.lyrics_btn = QPushButton("Select Lyrics File")
self.lyrics_btn.clicked.connect(self.select_lyrics)
layout.addWidget(self.lyrics_btn)

self.run_btn = QPushButton("Run Analysis")
self.run_btn.clicked.connect(self.run_analysis)
layout.addWidget(self.run_btn)

self.result_label = QLabel("Result: (no analysis yet)")
self.result_label.setAlignment(Qt.AlignLeft | Qt.AlignTop)
layout.addWidget(self.result_label)

self.setLayout(layout)

def select_audio(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Select Audio File", "", "Audio Files (*.wav *.mp3
*.flac);;All Files (*)"
    )
    if path:
        self.audio_path = path
        self.audio_btn.setText(f"Audio: {path}")

def select_lyrics(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Select Lyrics File", "", "Text Files (*.txt);;All Files
(*)"
    )
    if path:
        self.lyrics_path = path
        self.lyrics_btn.setText(f"Lyrics: {path}")

def run_analysis(self):
    try:
        if not self.audio_path or not self.lyrics_path:

```

```

        QMessageBox.warning(self, "Missing Files", "Please select
both audio and lyrics files.")
        return

    meta = SongMetadata(
        title=self.title_edit.text() or "Untitled",
        artist=self.artist_edit.text() or "Unknown",
        country=self.country_edit.text() or "Unknown",
        language=self.language_edit.text() or "Unknown",
        duration_seconds=float(self.duration_edit.text() or 180),
    )

    result = analyse_song(
        audio_path=self.audio_path,
        lyrics_path=self.lyrics_path,
        metadata=meta,
    )

    text = (
        f"Final Score: {result.scores.final_weighted_score:.2f}\n"
        f"Jury Score: {result.prediction.jury_score:.2f}\n"
        f"Televote Score: {result.prediction.televote_score:.2f}\n"
        f"Win Probability: {result.prediction.win_probability:.2%}"
    )
    self.result_label.setText(text)
except Exception as e:
    QMessageBox.critical(self, "Error", str(e))

```

main.py (desktop entry point)

python

```

from PySide6.QtWidgets import QApplication
import sys
from desktop_app.ui_main import MainWindow

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

This is now:

- A **complete backend engine** (Option 1)
- A **simple but working desktop app** (Option 2)
- A **CLI** for batch runs

A programmer can:

1. Create this folder structure
2. Paste each file as-is

3. Run:

bash

```
pip install -r requirements.txt  
python main.py
```

CHAPTER 21 — Program Bolt-ons

1. Updated `requirements.txt`

```
text
PySide6
librosa
soundfile
numpy
transformers
torch
textstat
```

A dev can install with: `pip install -r requirements.txt`

2. Upgraded `audio.py` (real DSP with `librosa`)

Replace your existing `eurovision_ai/audio.py` with this:

```
python
# evurovision_ai/audio.py
from typing import List
from .models import AudioFeatures
import os
import numpy as np
import librosa

def _load_audio(audio_path: str):
    if not os.path.exists(audio_path):
        raise FileNotFoundError(f"Audio file not found: {audio_path}")
    y, sr = librosa.load(audio_path, sr=None, mono=True)
    return y, sr

def extract_tempo(audio_path: str) -> float:
    y, sr = _load_audio(audio_path)
    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
    return float(tempo)

def extract_energy(audio_path: str) -> float:
    y, sr = _load_audio(audio_path)
    rms = librosa.feature.rms(y=y)[0]
    return float(np.clip(rms.mean(), 0.0, 1.0))

def extract_spectral_centroid(audio_path: str) -> float:
    y, sr = _load_audio(audio_path)
    centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]
    return float(centroid.mean())

def extract_chroma(audio_path: str) -> List[float]:
    y, sr = _load_audio(audio_path)
    chroma = librosa.feature.chroma_stft(y=y, sr=sr)
    return chroma.mean(axis=1).tolist()
```

```
def analyse_audio(audio_path: str) -> AudioFeatures:
    tempo = extract_tempo(audio_path)
    energy = extract_energy(audio_path)
    centroid = extract_spectral_centroid(audio_path)
    chroma = extract_chroma(audio_path)
    return AudioFeatures(
        tempo=tempo,
        energy=energy,
        spectral_centroid=centroid,
        chroma_vector=chroma,
    )
```

3. Upgraded lyrics.py (transformers + textstat)

Replace eurovision_ai/lyrics.py with:

```
python
# eurovision_ai/lyrics.py
from .models import LyricsFeatures
import os
from transformers import pipeline
import textstat

# Load once at module import (a dev can move this into an init step if
needed)
_sentiment_pipe = pipeline("sentiment-analysis", model="distilbert-base-
uncased-finetuned-sst-2-english")

def load_lyrics_text(path: str) -> str:
    if not os.path.exists(path):
        raise FileNotFoundError(f"Lyrics file not found: {path}")
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        return f.read()

def estimate_sentiment(text: str) -> float:
    if not text.strip():
        return 0.5
    result = _sentiment_pipe(text[:4000])[0] # truncate long texts
    score = result["score"]
    if result["label"].upper().startswith("NEG"):
        score = 1.0 - score
    return float(score)

def estimate_clarity(text: str) -> float:
    if not text.strip():
        return 0.5
    # Flesch Reading Ease: higher is easier
    fre = textstat.flesch_reading_ease(text)
    # Roughly map 0-100 → 0-1
    return float(max(0.0, min(1.0, fre / 100.0)))

def estimate_universality(text: str) -> float:
    if not text.strip():
        return 0.5
```

```

lower = text.lower()
# crude penalty for heavy local references
local_markers = [
    "euro", "brexit", "london", "paris", "berlin", "eiffel",
    "thames", "broadway", "hollywood", "instagram", "tiktok",
]
penalty = sum(1 for w in local_markers if w in lower)
base = 0.9
score = base - 0.05 * penalty
return float(max(0.0, min(1.0, score)))

def analyse_lyrics(lyrics_path: str) -> LyricsFeatures:
    text = load_lyrics_text(lyrics_path)
    sentiment = estimate_sentiment(text)
    clarity = estimate_clarity(text)
    universality = estimate_universality(text)
    return LyricsFeatures(
        sentiment=sentiment,
        clarity=clarity,
        universality=universality,
    )

```

4. Slightly smarter `scoring.py` (using real features)

Replace `eurovision_ai/scoring.py` with:

```

python
# evurovision_ai/scoring.py
from .models import SongFeatures, SongScores

WEIGHTS = {
    "structure": 0.15,
    "vocals": 0.2,
    "lyrics": 0.2,
    "originality": 0.15,
    "staging_potential": 0.1,
    "universality": 0.1,
    "replay_value": 0.1,
}

def _norm(value: float, min_v: float, max_v: float) -> float:
    if max_v == min_v:
        return 0.5
    x = (value - min_v) / (max_v - min_v)
    return float(max(0.0, min(1.0, x)))

def score_structure(features: SongFeatures) -> float:
    tempo = features.audio.tempo
    # favour mid-tempo for "balanced structure"
    return 1.0 - abs(tempo - 120.0) / 120.0 if tempo > 0 else 0.5

def score_vocals(features: SongFeatures) -> float:
    # still a stub - dev can plug in vocal isolation later
    return 0.8

```



```

def score_lyrics(features: SongFeatures) -> float:
    return (features.lyrics.sentiment + features.lyrics.clarity) / 2.0

def score_originality(features: SongFeatures) -> float:
    # crude: high spectral centroid + non-extreme tempo → more
    "modern/original"
    t = _norm(features.audio.tempo, 60, 180)
    c = _norm(features.audio.spectral_centroid, 1000, 6000)
    return (t + c) / 2.0

def score_staging_potential(features: SongFeatures) -> float:
    return features.audio.energy

def score_universality(features: SongFeatures) -> float:
    return features.lyrics.universality

def score_replay_value(features: SongFeatures) -> float:
    # high energy + positive sentiment → more replayable
    return (features.audio.energy + features.lyrics.sentiment) / 2.0

def compute_scores(features: SongFeatures) -> SongScores:
    structure = score_structure(features)
    vocals = score_vocals(features)
    lyrics = score_lyrics(features)
    originality = score_originality(features)
    staging = score_staging_potential(features)
    universality = score_universality(features)
    replay = score_replay_value(features)

    final = (
        structure * WEIGHTS["structure"]
        + vocals * WEIGHTS["vocals"]
        + lyrics * WEIGHTS["lyrics"]
        + originality * WEIGHTS["originality"]
        + staging * WEIGHTS["staging_potential"]
        + universality * WEIGHTS["universality"]
        + replay * WEIGHTS["replay_value"]
    )

    return SongScores(
        structure=structure,
        vocals=vocals,
        lyrics=lyrics,
        originality=originality,
        staging_potential=staging,
        universality=universality,
        replay_value=replay,
        final_weighted_score=final,
    )

```

5. Prediction engine: keep simple, but now driven by real scores

You can keep `prediction.py` as-is; it already uses the scores meaningfully. A dev can later swap in a trained model.

6. How a dev runs the whole thing

Desktop app:

```
bash
pip install -r requirements.txt
python main.py
```

CLI (batch):

```
bash
python cli.py --config songs_config.json --output results.json
```

Where songs_config.json looks like:

```
json
{
  "songs": {
    "Song A": {
      "audio": "path/to/song_a.wav",
      "lyrics": "path/to/song_a.txt",
      "metadata": {
        "title": "Song A",
        "artist": "Artist A",
        "country": "Country A",
        "language": "English",
        "duration_seconds": 180
      }
    }
  }
}
```

MULTI-SONG VIEW — FULL BOLT-ON IMPLEMENTATION

This adds a new screen to your PySide6 desktop app:

- A **Song List Table**
- A **Load Multiple Songs** button
- A **Run Analysis for All** button
- A **Results Table**
- A **Song Detail Popup**

Everything below is **copy-paste ready**.

New File: `desktop_app/ui_multisong.py`

python

```
# desktop_app/ui_multisong.py
from PySide6.QtWidgets import (
    QWidget, QVBoxLayout, QPushButton, QTableWidget, QTableWidgetItem,
    QFileDialog, QMessageBox, QHBoxLayout, QLabel, QDialog
)
from PySide6.QtCore import Qt
from eurovision_ai.models import SongMetadata
from eurovision_ai.pipeline import analyse_song

class SongDetailDialog(QDialog):
    def __init__(self, result, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Song Details")
        layout = QVBoxLayout()

        text = (
            f"Title: {result.features.metadata.title}\n"
            f"Artist: {result.features.metadata.artist}\n"
            f"Country: {result.features.metadata.country}\n"
            f"Language: {result.features.metadata.language}\n"
            f"Duration: {result.features.metadata.duration_seconds}s\n\n"
            f"Final Score: {result.scores.final_weighted_score:.2f}\n"
            f"Jury Score: {result.prediction.jury_score:.2f}\n"
            f"Televote Score: {result.prediction.televote_score:.2f}\n"
            f"Win Probability: {result.prediction.win_probability:.2%}"
        )

        label = QLabel(text)
        label.setAlignment(Qt.AlignLeft | Qt.AlignTop)
        layout.addWidget(label)

        self.setLayout(layout)

class MultiSongView(QWidget):
    def __init__(self):
```

```

super().__init__()
self.setWindowTitle("Eurovision Multi-Song Analysis")

self.songs = [] # list of dicts: {audio, lyrics, metadata}
self.results = {}

layout = QVBoxLayout()

# Buttons
btn_row = QHBoxLayout()
self.load_btn = QPushButton("Load Songs")
self.load_btn.clicked.connect(self.load_songs)
btn_row.addWidget(self.load_btn)

self.run_btn = QPushButton("Run Analysis for All")
self.run_btn.clicked.connect(self.run_all)
btn_row.addWidget(self.run_btn)

layout.addLayout(btn_row)

# Table
self.table = QTableWidgetItem(0, 5)
self.table.setHorizontalHeaderLabels(
    ["Title", "Artist", "Country", "Final Score", "Win %"]
)
self.table.cellDoubleClicked.connect(self.show_details)
layout.addWidget(self.table)

self.setLayout(layout)

def load_songs(self):
    audio_files, _ = QFileDialog.getOpenFileNames(
        self, "Select Audio Files", "", "Audio (*.wav *.mp3 *.flac)"
    )
    if not audio_files:
        return

    lyrics_files, _ = QFileDialog.getOpenFileNames(
        self, "Select Lyrics Files", "", "Text (*.txt)"
    )
    if not lyrics_files or len(lyrics_files) != len(audio_files):
        QMessageBox.warning(
            self, "Mismatch", "You must select the same number of
lyrics files."
        )
        return

    self.songs = []
    for audio, lyrics in zip(audio_files, lyrics_files):
        title = audio.split("/")[-1].split(".")[0]
        meta = SongMetadata(
            title=title,
            artist="Unknown",
            country="Unknown",
            language="Unknown",
            duration_seconds=180,
        )
        self.songs.append({"audio": audio, "lyrics": lyrics,
"metadata": meta})

    self.refresh_table()

```

```

def refresh_table(self):
    self.table.setRowCount(len(self.songs))
    for i, s in enumerate(self.songs):
        self.table.setItem(i, 0, QTableWidgetItem(s["metadata"].title))
        self.table.setItem(i, 1,
QTableWidgetItem(s["metadata"].artist))
        self.table.setItem(i, 2,
QTableWidgetItem(s["metadata"].country))
        self.table.setItem(i, 3, QTableWidgetItem("-"))
        self.table.setItem(i, 4, QTableWidgetItem("-"))

def run_all(self):
    if not self.songs:
        QMessageBox.warning(self, "No Songs", "Load songs first.")
        return

    for i, s in enumerate(self.songs):
        result = analyse_song(
            audio_path=s["audio"],
            lyrics_path=s["lyrics"],
            metadata=s["metadata"],
        )
        self.results[i] = result

        self.table.setItem(
            i, 3,
QTableWidgetItem(f"{result.scores.final_weighted_score:.2f}")
        )
        self.table.setItem(
            i, 4,
QTableWidgetItem(f"{result.prediction.win_probability:.2%}")
        )

        QMessageBox.information(self, "Done", "Analysis complete.")

def show_details(self, row, col):
    if row not in self.results:
        QMessageBox.warning(self, "Not analysed", "Run analysis
first.")
        return

    dlg = SongDetailDialog(self.results[row], self)
    dlg.exec()

```

Update `main.py` to include Multi-Song View

Replace your `main.py` with:

python

```

from PySide6.QtWidgets import QApplication, QWidget, QVBoxLayout,
QPushButton
import sys
from desktop_app.ui_main import MainWindow
from desktop_app.ui_multisong import MultiSongView

```

```

class Launcher(QWidget):

```

```

def __init__(self):
    super().__init__()
    self.setWindowTitle("Eurovision AI Launcher")

    layout = QVBoxLayout()

    btn_single = QPushButton("Single Song Analysis")
    btn_single.clicked.connect(self.open_single)
    layout.addWidget(btn_single)

    btn_multi = QPushButton("Multi-Song Analysis")
    btn_multi.clicked.connect(self.open_multi)
    layout.addWidget(btn_multi)

    self.setLayout(layout)

def open_single(self):
    self.win = MainWindow()
    self.win.show()

def open_multi(self):
    self.win = MultiSongView()
    self.win.show()

def main():
    app = QApplication(sys.argv)
    launcher = Launcher()
    launcher.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

Step 1 — Folder structure, README, and requirements

■ Folder tree (what you'll create)

text

```
eurovision_ai/  
  main.py  
  README.md  
  requirements.txt  
  eurovision_core/  
    __init__.py  
    config.py  
    models.py  
    audio_dsp.py  
    lyrics_nlp.py  
    scoring.py  
    pipeline.py  
    storage.py  
  eurovision_ui/  
    __init__.py  
    launcher.py  
    single_song_view.py  
    multi_song_view.py
```

Create that folder layout first.

README.md

Create `README.md` in the root with this content:

markdown

```
# Eurovision AI - Song Analysis Toolkit  
  
This project analyzes Eurovision-style songs using:  
- Audio DSP (librosa)  
- Lyrics NLP (HuggingFace transformers)  
- A desktop UI (PySide6)  
- A simple pipeline and scoring engine  
  
## 1. Installation  
  
```bash  
python -m venv .venv
Activate your venv (Windows: .venv\Scripts\activate, macOS/Linux: source
.venv/bin/activate)

pip install -r requirements.txt
```

## 2. Running the app

bash

```
python main.py
```

The launcher window will open, letting you:

- Analyze a single song (audio + lyrics)
- Open the multi-song dashboard

### 3. Project structure

- `main.py` – entry point, starts the UI
- `eurovision_core/` – backend logic
  - `config.py` – configuration constants and paths
  - `models.py` – core data models (Song, AnalysisResult, etc.)
  - `audio_dsp.py` – audio feature extraction (librosa)
  - `lyrics_nlp.py` – lyrics analysis (transformers, textstat)
  - `scoring.py` – combines DSP + NLP into scores
  - `pipeline.py` – orchestration functions
  - `storage.py` – simple JSON-based persistence
- `eurovision_ui/` – PySide6 UI
  - `launcher.py` – main launcher window
  - `single_song_view.py` – single-song analysis UI
  - `multi_song_view.py` – multi-song dashboard

### 4. Adding songs

You can:

- Point the UI at an audio file + lyrics text
- Or extend `storage.py` to load/save song metadata in bulk

### 5. Extending the system

- Add new DSP features in `audio_dsp.py`
- Add new NLP metrics in `lyrics_nlp.py`
- Adjust scoring weights in `scoring.py`
- Add new views or controls in `eurovision_ui/`

Code

---

#### 📄 requirements.txt

Create `requirements.txt` in the root:

```
```text
PySide6==6.7.0
librosa==0.10.2.post1
numpy==1.26.4
soundfile==0.12.1
transformers==4.40.0
torch==2.2.2
textstat==0.7.4
```


(Your programmer can tweak versions if needed.)

main.py

Create `main.py` in the root:

python

```
import sys
from PySide6.QtWidgets import QApplication
from eurovision_ui.launcher import LauncherWindow
```

```
def main():
    app = QApplication(sys.argv)
    window = LauncherWindow()
    window.show()
    sys.exit(app.exec())
```

```
if __name__ == "__main__":
    main()
```

1□ Create the folder structure

In your project root, create this:

text

```
eurovision_ai/
  main.py
  README.md
  requirements.txt
eurovision_core/
  __init__.py
  config.py
  models.py
  audio_dsp.py
  lyrics_nlp.py
  scoring.py
  pipeline.py
  storage.py
eurovision_ui/
  __init__.py
  launcher.py
  single_song_view.py
  multi_song_view.py
```

You don't need to fill the files yet—just create empty ones with those names.

2□ README.md

Paste this into `README.md`:

markdown

```
# Eurovision AI - Song Analysis Toolkit
```

This project analyzes Eurovision-style songs using:

- Audio DSP (librosa)
- Lyrics NLP (HuggingFace transformers)
- A desktop UI (PySide6)
- A simple pipeline and scoring engine

1. Installation

```
```bash
python -m venv .venv
Activate your venv:
Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate
```

```
pip install -r requirements.txt
```

## 2. Running the app

```
bash
python main.py
```

The launcher window will open, letting you:

- Analyze a single song (audio + lyrics)
- Open the multi-song dashboard

## 3. Project structure

- `main.py` – entry point, starts the UI
- `eurovision_core/` – backend logic
  - `config.py` – configuration constants and paths
  - `models.py` – core data models (Song, AnalysisResult, etc.)
  - `audio_dsp.py` – audio feature extraction (librosa)
  - `lyrics_nlp.py` – lyrics analysis (transformers, textstat)
  - `scoring.py` – combines DSP + NLP into scores
  - `pipeline.py` – orchestration functions
  - `storage.py` – simple JSON-based persistence
- `eurovision_ui/` – PySide6 UI
  - `launcher.py` – main launcher window
  - `single_song_view.py` – single-song analysis UI
  - `multi_song_view.py` – multi-song dashboard

## 4. Adding songs

You can:

- Point the UI at an audio file + lyrics text
- Or extend `storage.py` to load/save song metadata in bulk

## 5. Extending the system

- Add new DSP features in `audio_dsp.py`
- Add new NLP metrics in `lyrics_nlp.py`
- Adjust scoring weights in `scoring.py`
- Add new views or controls in `eurovision_ui/`

## Code

```

3 requirements.txt

Paste this into `requirements.txt`:
```

```
```text
PySide6==6.7.0
librosa==0.10.2.post1
numpy==1.26.4
soundfile==0.12.1
transformers==4.40.0
torch==2.2.2
textstat==0.7.4
```

4 main.py

Paste this into `main.py`:

```
python
import sys
from PySide6.QtWidgets import QApplication
from eurovision_ui.launcher import LauncherWindow

def main():
    app = QApplication(sys.argv)
    window = LauncherWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

SECTION 2 — BACKEND CORE

(`eurovision_core/`)

Create or replace the following files exactly as shown.

`eurovision_core/__init__.py`

python

```
# empty on purpose
```

`eurovision_core/config.py`

python

```
import os

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

MODEL_NAME = "distilbert-base-uncased-finetuned-sst-2-english"

CACHE_DIR = os.path.join(BASE_DIR, "cache")
os.makedirs(CACHE_DIR, exist_ok=True)
```

`eurovision_core/models.py`

python

```
from dataclasses import dataclass
from typing import Dict, Any

@dataclass
class Song:
    title: str
    artist: str
    audio_path: str
    lyrics: str

@dataclass
class AudioFeatures:
    tempo: float
    energy: float
    spectral_centroid: float
    chroma_mean: float
```

```

@dataclass
class LyricsFeatures:
    sentiment: float
    readability: float
    complexity: float

@dataclass
class AnalysisResult:
    audio: AudioFeatures
    lyrics: LyricsFeatures
    final_score: float
    details: Dict[str, Any]

```

eurovision_core/audio_dsp.py

python

```

import librosa
import numpy as np
from .models import AudioFeatures

def extract_audio_features(path: str) -> AudioFeatures:
    y, sr = librosa.load(path, sr=None)

    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)

    energy = float(np.mean(y ** 2))

    spectral_centroid =
float(np.mean(librosa.feature.spectral_centroid(y=y, sr=sr)))

    chroma = librosa.feature.chroma_stft(y=y, sr=sr)
    chroma_mean = float(np.mean(chroma))

    return AudioFeatures(
        tempo=tempo,
        energy=energy,
        spectral_centroid=spectral_centroid,
        chroma_mean=chroma_mean,
    )

```

eurovision_core/lyrics_nlp.py

python

```

from transformers import pipeline
import textstat
from .models import LyricsFeatures
from .config import MODEL_NAME

_sentiment = pipeline("sentiment-analysis", model=MODEL_NAME)

```

```

def extract_lyrics_features(text: str) -> LyricsFeatures:
    result = _sentiment(text[:512])[0] # limit to safe length

    sentiment_score = result["score"] if result["label"] == "POSITIVE" else
    -result["score"]

    readability = textstat.flesch_reading_ease(text)
    complexity = textstat.text_standard(text, float_output=True)

    return LyricsFeatures(
        sentiment=float(sentiment_score),
        readability=float(readability),
        complexity=float(complexity),
    )

```

eurovision_core/scoring.py

python

```

from .models import AudioFeatures, LyricsFeatures

def compute_final_score(audio: AudioFeatures, lyrics: LyricsFeatures) ->
float:
    score = 0

    score += (audio.tempo / 200) * 20
    score += audio.energy * 15
    score += (audio.spectral_centroid / 4000) * 15
    score += audio.chroma_mean * 10

    score += (lyrics.sentiment + 1) * 10
    score += (100 - lyrics.readability) * 5
    score += lyrics.complexity * 5

    return max(0, min(100, score))

```

eurovision_core/pipeline.py

python

```

from .audio_dsp import extract_audio_features
from .lyrics_nlp import extract_lyrics_features
from .scoring import compute_final_score
from .models import Song, AnalysisResult

def analyze_song(song: Song) -> AnalysisResult:
    audio = extract_audio_features(song.audio_path)
    lyrics = extract_lyrics_features(song.lyrics)
    final_score = compute_final_score(audio, lyrics)

    return AnalysisResult(
        audio=audio,

```

```

        lyrics=lyrics,
        final_score=final_score,
        details={
            "audio": audio.__dict__,
            "lyrics": lyrics.__dict__,
        },
    )

```

eurovision_core/storage.py

python

```

import json
import os
from typing import List
from .models import Song

STORAGE_FILE = "songs.json"

def save_songs(songs: List[Song]):
    data = [song.__dict__ for song in songs]
    with open(STORAGE_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4)

def load_songs() -> List[Song]:
    if not os.path.exists(STORAGE_FILE):
        return []

    with open(STORAGE_FILE, "r", encoding="utf-8") as f:
        data = json.load(f)

    return [Song(**entry) for entry in data]

```

SECTION 3 — UI LAYER (`eurovision_ui/`)

This gives you:

- The launcher window
- The single-song analysis UI
- The multi-song dashboard

All built with **PySide6**, matching the backend you already pasted.

`eurovision_ui/__init__.py`

python

```
# empty on purpose
```

`eurovision_ui/launcher.py`

python

```
from PySide6.QtWidgets import QWidget, QPushButton, QVBoxLayout
from .single_song_view import SingleSongWindow
from .multi_song_view import MultiSongWindow
```

```
class LauncherWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Eurovision AI - Launcher")
        self.setMinimumWidth(300)

        layout = QVBoxLayout()

        btn_single = QPushButton("Analyze Single Song")
        btn_multi = QPushButton("Open Multi-Song Dashboard")

        btn_single.clicked.connect(self.open_single)
        btn_multi.clicked.connect(self.open_multi)

        layout.addWidget(btn_single)
        layout.addWidget(btn_multi)

        self.setLayout(layout)

    def open_single(self):
        self.win = SingleSongWindow()
        self.win.show()

    def open_multi(self):
        self.win = MultiSongWindow()
```



```
self.win.show()
```

eurovision_ui/single_song_view.py

python

```
from PySide6.QtWidgets import (
    QWidget, QVBoxLayout, QLabel, QPushButton,
    QFileDialog, QTextEdit, QLineEdit
)
from eurovision_core.models import Song
from eurovision_core.pipeline import analyze_song

class SingleSongWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Single Song Analysis")
        self.audio_path = None

        layout = QVBoxLayout()

        self.title_input = QLineEdit()
        self.title_input.setPlaceholderText("Song Title")

        self.artist_input = QLineEdit()
        self.artist_input.setPlaceholderText("Artist")

        self.lyrics_box = QTextEdit()
        self.lyrics_box.setPlaceholderText("Paste lyrics here...")

        self.btn_audio = QPushButton("Choose Audio File")
        self.btn_audio.clicked.connect(self.choose_audio)

        self.btn_analyze = QPushButton("Analyze")
        self.btn_analyze.clicked.connect(self.run_analysis)

        self.output = QLabel("Results will appear here.")

        layout.addWidget(self.title_input)
        layout.addWidget(self.artist_input)
        layout.addWidget(self.lyrics_box)
        layout.addWidget(self.btn_audio)
        layout.addWidget(self.btn_analyze)
        layout.addWidget(self.output)

        self.setLayout(layout)

    def choose_audio(self):
        path, _ = QFileDialog.getOpenFileName(self, "Select Audio", "",
        "Audio Files (*.mp3 *.wav)")
        if path:
            self.audio_path = path
            self.btn_audio.setText(f"Audio Selected")

    def run_analysis(self):
        if not self.audio_path:
```

```

        self.output.setText("Please select an audio file.")
        return

    song = Song(
        title=self.title_input.text(),
        artist=self.artist_input.text(),
        audio_path=self.audio_path,
        lyrics=self.lyrics_box.toPlainText(),
    )

    result = analyze_song(song)

    self.output.setText(
        f"<b>Final Score:</b> {result.final_score:.2f}<br>"
        f"<b>Tempo:</b> {result.audio.tempo:.2f}<br>"
        f"<b>Energy:</b> {result.audio.energy:.4f}<br>"
        f"<b>Sentiment:</b> {result.lyrics.sentiment:.3f}<br>"
        f"<b>Readability:</b> {result.lyrics.readability:.2f}<br>"
    )

```

eurovision_ui/multi_song_view.py

python

```

from PySide6.QtWidgets import (
    QWidget, QVBoxLayout, QPushButton, QListWidget, QLabel, QFileDialog
)
from eurovision_core.storage import load_songs, save_songs
from eurovision_core.pipeline import analyze_song
from eurovision_core.models import Song

class MultiSongWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Multi-Song Dashboard")

        layout = QVBoxLayout()

        self.list = QListWidget()
        self.btn_add = QPushButton("Add Song")
        self.btn_analyze = QPushButton("Analyze All")
        self.output = QLabel("Results will appear here.")

        self.btn_add.clicked.connect(self.add_song)
        self.btn_analyze.clicked.connect(self.analyze_all)

        layout.addWidget(self.list)
        layout.addWidget(self.btn_add)
        layout.addWidget(self.btn_analyze)
        layout.addWidget(self.output)

        self.setLayout(layout)

        self.songs = load_songs()
        self.refresh_list()

```

```

def refresh_list(self):
    self.list.clear()
    for s in self.songs:
        self.list.addItem(f"{s.title} - {s.artist}")

def add_song(self):
    audio_path, _ = QFileDialog.getOpenFileName(self, "Select Audio",
        "", "Audio Files (*.mp3 *.wav)")
    if not audio_path:
        return

    lyrics_path, _ = QFileDialog.getOpenFileName(self, "Select Lyrics",
        "", "Text Files (*.txt)")
    if not lyrics_path:
        return

    with open(lyrics_path, "r", encoding="utf-8") as f:
        lyrics = f.read()

    title = audio_path.split("/")[-1]
    artist = "Unknown"

    song = Song(title=title, artist=artist, audio_path=audio_path,
        lyrics=lyrics)
    self.songs.append(song)
    save_songs(self.songs)
    self.refresh_list()

def analyze_all(self):
    if not self.songs:
        self.output.setText("No songs to analyze.")
        return

    results = []
    for s in self.songs:
        res = analyze_song(s)
        results.append((s.title, res.final_score))

    results.sort(key=lambda x: x[1], reverse=True)

    text = "<b>Rankings:</b><br>"
    for title, score in results:
        text += f"{title}: {score:.2f}<br>"

    self.output.setText(text)

```

SECTION 4 — CLI (`cli.py`)

Create a new file in your project root:

Code

```
eurovision_ai/  
  cli.py ← this one
```

Then paste this inside:

python

```
import argparse  
from eurovision_core.models import Song  
from eurovision_core.pipeline import analyze_song  
from eurovision_core.storage import load_songs, save_songs  
  
def cmd_analyze(args):  
    with open(args.lyrics, "r", encoding="utf-8") as f:  
        lyrics = f.read()  
  
    song = Song(  
        title=args.title,  
        artist=args.artist,  
        audio_path=args.audio,  
        lyrics=lyrics,  
    )  
  
    result = analyze_song(song)  
  
    print(f"Final Score: {result.final_score:.2f}")  
    print(f"Tempo: {result.audio.tempo:.2f}")  
    print(f"Energy: {result.audio.energy:.4f}")  
    print(f"Sentiment: {result.lyrics.sentiment:.3f}")  
    print(f"Readability: {result.lyrics.readability:.2f}")  
  
def cmd_batch(args):  
    songs = load_songs()  
    if not songs:  
        print("No songs saved.")  
        return  
  
    results = []  
    for s in songs:  
        res = analyze_song(s)  
        results.append((s.title, res.final_score))  
  
    results.sort(key=lambda x: x[1], reverse=True)  
  
    print("Rankings:")  
    for title, score in results:  
        print(f"{title}: {score:.2f}")  
  
def main():
```

```

parser = argparse.ArgumentParser(description="Eurovision AI CLI")
sub = parser.add_subparsers()

# analyze command
p_analyze = sub.add_parser("analyze", help="Analyze a single song")
p_analyze.add_argument("--audio", required=True, help="Path to audio
file")
p_analyze.add_argument("--lyrics", required=True, help="Path to lyrics
text file")
p_analyze.add_argument("--title", required=True, help="Song title")
p_analyze.add_argument("--artist", required=True, help="Artist name")
p_analyze.set_defaults(func=cmd_analyze)

# batch command
p_batch = sub.add_parser("batch", help="Analyze all saved songs")
p_batch.set_defaults(func=cmd_batch)

args = parser.parse_args()
if hasattr(args, "func"):
    args.func(args)
else:
    parser.print_help()

if __name__ == "__main__":
    main()

```

How to use the CLI

Analyze a single song:

Code

```
python cli.py analyze --audio song.wav --lyrics lyrics.txt --title "My
Song" --artist "Me"
```

Analyze all saved songs:

Code

```
python cli.py batch
```

Project Introduction (ready to use)

Eurovision AI Song Analysis Toolkit *An open, modular blueprint for building intelligent Eurovision-style music analysis tools.*

The Eurovision AI Song Analysis Toolkit is a **free, open technical manual** designed so that any programmer — beginner or advanced — can implement, extend, or reinvent the system however they wish. It is not a finished application. Instead, it is a **complete architectural guide**, covering audio analysis, lyrical evaluation, scoring logic, UI design, and CLI tooling.

The goal is simple: **Give developers everything they need to build their own Eurovision-style song analysis engine from scratch.**

This manual provides:

- A clear folder structure (`eurovision_core`, `eurovision_ui`, `cli.py`, `main.py`)
- Detailed module responsibilities
- Audio processing using `librosa`
- NLP pipelines using `transformers` and readability metrics
- A scoring framework that can be customised or replaced
- Optional features such as charts, exports, and UI styling
- A roadmap for future contributors

Anyone is free to:

- Implement the toolkit exactly as described
- Fork it and build their own version
- Add new models, metrics, or visualisations
- Create plugins or alternative UIs
- Turn it into something bigger, stranger, or more Eurovision than ever

This project is intentionally open, flexible, and community-driven. If you build something with it, improve it, or remix it — that's the point.

Licence (MIT Licence — recommended for open, flexible projects)

Here is a clean, ready-to-paste MIT Licence:

Code

MIT License

Copyright (c) 2026 Paul Williams

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.